

Today's Content

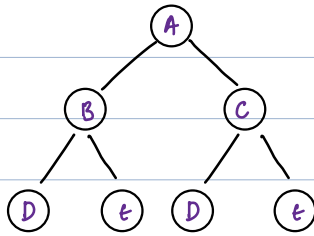
1. Heaps Intro
2. Insert/delete()
3. heapify()
4. Problems on heap

Saturday : 7:00AM

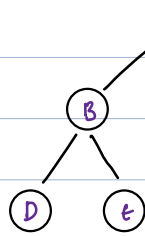
CBT Complete Binary Tree.

All levels are filled [except possibly] the last level,
which is filled left $\rightarrow$ right.

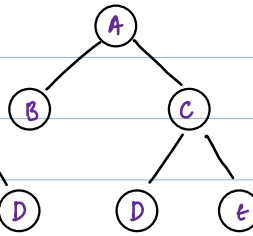
Ex1: CBT



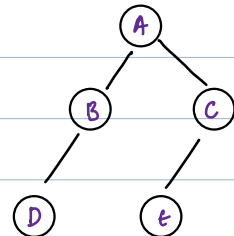
Ex2: Not CBT



Ex3: Not CBT



Ex4: Not CBT

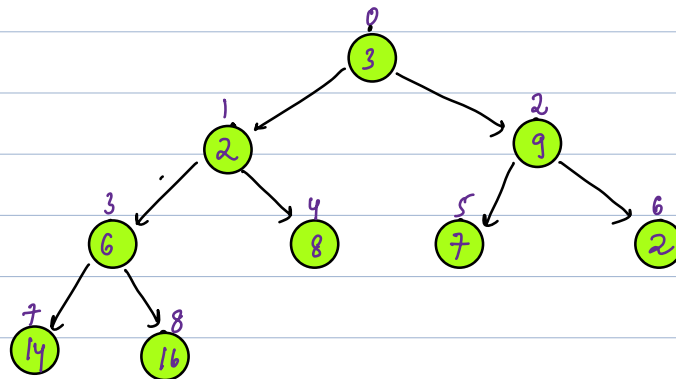


Height of CBT: $\log_2 N$, $N = \text{No. of Nodes}$. $\rightarrow$ [Stay back]

Implementation of CBT using arrays/Link

Insert: 3 2 9 6 8 7 2 14 16 $\rightarrow$ Implement a CBT:
list: 3 2 9 6 8 7 2 14 16

Parent	left	Right
0	1	2
1	3	4
2	5	6
3	7	8
i	$2i+1$	$2i+2$



obs:

parent i : $2i+1$ $2i+2$

child i : $(i-1)/2$

Advantage:

parent index $\rightarrow$ We can access child index

child index $\rightarrow$ We can access parent index

It allows both Top down & down Top.

Heaps: a. MinHeap b. MaxHeap

1. BT should be a CBT first

2a. Every node $\leq$ child nodes

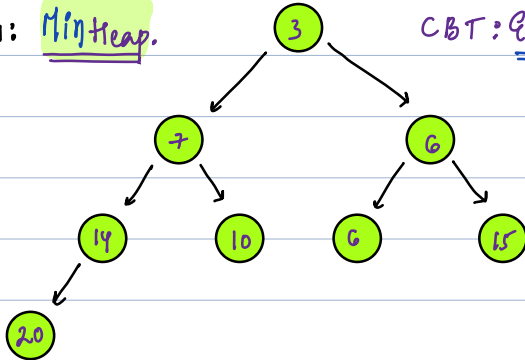
MinHeap

2b. Every node $\geq$ child nodes

MaxHeap

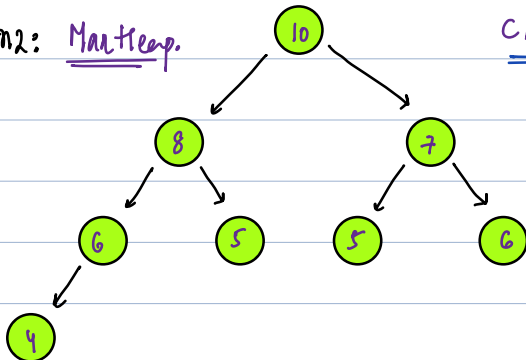
Ex1: MinHeap.

CBT: Every node $\leq$ child nodes



Ex2: MaxHeap.

CBT: Every node $\geq$ child nodes



Heap Operations.

MinHeap

insert()
getMin()
deleteMin()
size()
heapify()

MaxHeap

insert()
getMax()
deleteMax()
size()
heapify()

Time Complexity for Single

$O(\log N)$ // N: No of Nodes

$O(1)$

$O(\log N)$ // N: No of Nodes

$O(1)$

$O(N)$ // N: no of Nodes

Heapify: Using given data create MinHeap/MaxHeap

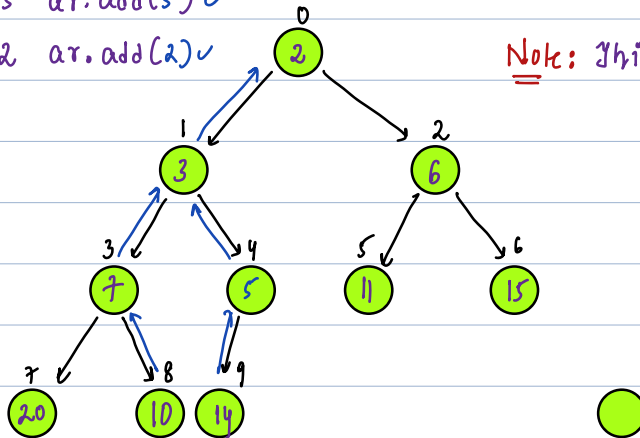
Given min heap :

list: ar	0	1	2	3	4	5	6	7	8	9
	2	3	6	7	5	11	15	20	10	14

Insert: 5 ar.add(5) ✓

Insert: 2 ar.add(2) ✓

Note: This tree is our imagination



$i-1/2$

Insert(5) :	ind	par	Ideally: $ar[par] \leq ar[ind]$
	8	3	$ar[3] \leq ar[8]$: swap $ar[3]$ & $ar[8]$
	3	1	$ar[1] \leq ar[3]$: swap $ar[1]$ & $ar[3]$
	1	0	$ar[0] \leq ar[1]$: <u>break it.</u>

$i-1/2$

Insert(2) :	ind	par	Ideally: $ar[par] \leq ar[ind]$
	9	4	$ar[4] \leq ar[9]$: swap $ar[4]$ & $ar[9]$
	4	1	$ar[1] \leq ar[4]$: swap $ar[1]$ & $ar[4]$
	1	0	$ar[0] \leq ar[1]$: swap $ar[0]$ & $ar[1]$

0 : at root, stop it.

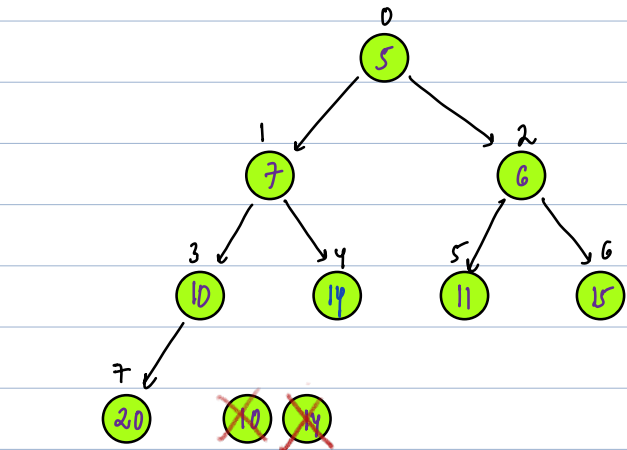
Note: for a single insert, we travel from leaf node to root node, dist = height

In BST height = $\log_2 N$, No. of nodes.

Delete - min() :

deleted.

0	1	2	3	4	5	6	7	8	9
5	8	10	6	7	14	11	15	20	10



Step: Copy last element to start & delete last.

	left	right	min_ind
ind	$2i+1$	$2i+2$	
0	arr[1]	arr[2]	arr[1], swap arr[0] & arr[1]
1	arr[3]	arr[4]	arr[4], swap arr[1] & arr[4]
4	97 = N : no child break up.		

ind	left	right	min_ind
	$2i+1$	$2i+2$	
0	arr[1]	arr[2]	arr[1], swap arr[0] & arr[1]
1	arr[3]	arr[4]	arr[3], swap arr[1] & arr[3]
3	arr[7]	no right	arr[7] $arr[3] \leq arr[7]$: MinHeap ✓ break.

Heapify(): Given an arr[N] of elements, convert them into MinHeap / MaxHeap

Ex: arr[12] = { 0 1 2 3 4 5 6 7 8 9 10 11 }
{ 6 2 8 10 14 3 9 2 7 20 1 -3 }

Idea: Create a empty minheap & insert each ele 1 by 1 in MinHeap. ✓
TC: $O(N \log N)$

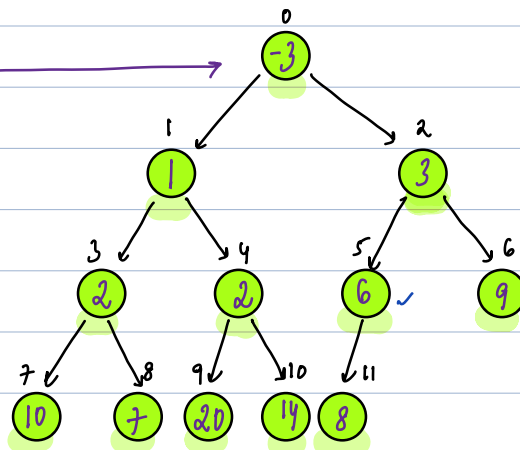
Idea2: Add arr elements in arraylist as it is & apply heapify?

Heapify: Iterate n right to left, at every index, make that entire subtree a min-heap by comparing ele with it's children.

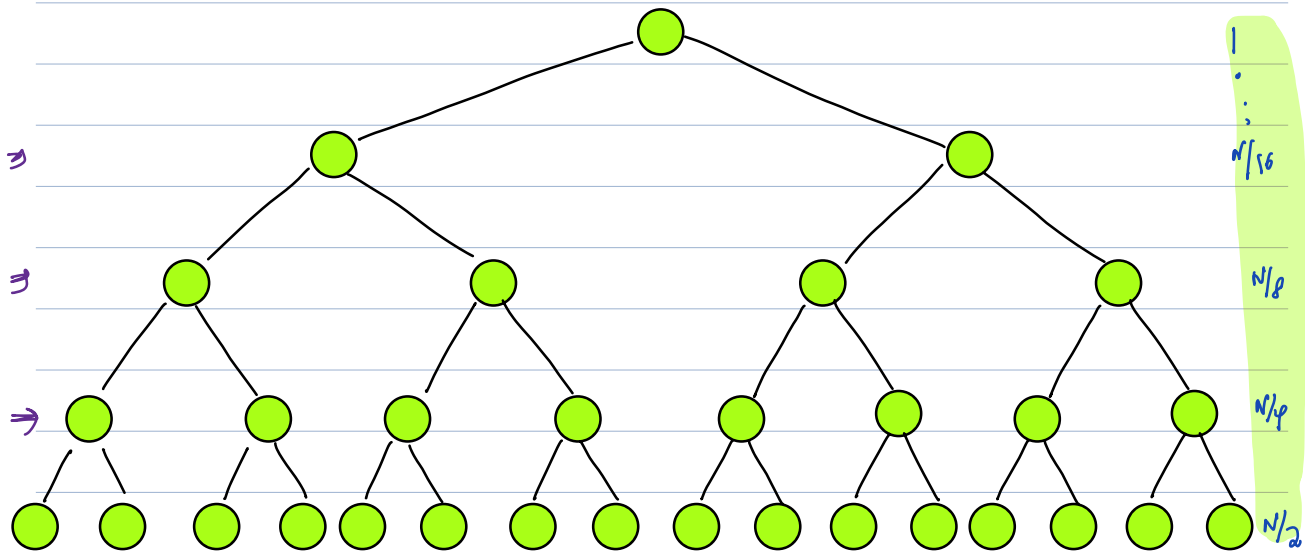
0 1 2 3 4 5 6 7 8 9 10 11
list: { 6 2 8 10 14 3 9 2 7 20 1 -3 }

After Heapify: { -3 1 3 2 2 6 9 10 7 20 14 8 }

Imaginary
Min Heap



Heapify TC: In CBT For N : Nodes: leaf Nodes = $N/2$
 Below is Approx CBT for N : Nodes



Total iteration $\left[\frac{N}{2}\right] \times 0 + \left[\frac{N}{4}\right] \times 1 + \left[\frac{N}{8}\right] \times 2 + \left[\frac{N}{16}\right] \times 3 \dots$

$$\frac{N}{4} + \frac{2N}{8} + \frac{3N}{16} + \dots$$

$$\frac{N}{2} \left[\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} \dots \right] = \frac{N}{2} [2] = N = O(N)$$

A.G.P

$$S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} \dots$$

$$2S = 1 + \left[\frac{2}{2}\right] + \left[\frac{3}{4}\right] + \left[\frac{4}{8}\right] + \left[\frac{5}{16}\right] \dots - \left[\frac{1}{2}\right] + \left[\frac{2}{4}\right] + \left[\frac{3}{8}\right] + \left[\frac{4}{16}\right] + \frac{5}{32} \dots$$

$$S = 1 + \left[\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} \dots\right] \approx G.P = \frac{a}{1-r} = \frac{\frac{1}{2}}{1-\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}} = 1$$

$$S = 1 + 1 = 2$$

```
class Min_Heap {
```

```
private ArrayList<Integer> ar;
```

```
Min_Heap() { // constructor
```

```
    ar = new ArrayList<Integer>();  
}
```

```
void delete_Min() { TC:  $O(\log N)$ 
```

```
    ar.set(0, ar.get(ar.size()-1)); // Copy last to first
```

```
    ar.remove(ar.size()-1); // last element removed.
```

```
    int N = ar.size();
```

```
    int i = 0;
```

```
    while(2*i+1 < N) { // At least 1 kid exists
```

```
        int min_ind = 2*i+1;
```

```
        if( (2*i+2 < N) && (ar[2*i+2] < ar[min_ind]) ) {
```

```
            min_ind = 2*i+2;  
        }
```

```
        if( ar[i] > ar[min_ind] ) {
```

```
            swap ar[i] & ar[min_ind];
```

```
            i = min_ind;
```

```
        } else { break; }
```

```
    }
```

```
int getMin() { TC:  $O(1)$ 
```

```
    return ar.get(0);  
}
```

```
int size() { TC:  $O(1)$ 
```

```
    return ar.size();  
}
```


void insert(int x) { Tc: $O(\log N)$

ar.add(x); // insert at last;

int ind = ar.size() - 1;

while(ind > 0) {

int par = (ind - 1) / 2;

if(ar[par] > ar[ind]) {

swap ar[par] & ar[ind]

ind = par;

else { // ar[par] < ar[ind]

break;

} } // stay back:

void heapify(int ar[]) { Tc: $O(N)$ { ar[N] $\rightarrow$ heap : Tc: $O(1)$ }

for(int i = 0; i < ar.length; i++) {

ar.add(ar[i]);

int N = ar.size();

for(int ind = N - 1; ind >= 0; ind--) {

int i = ind; // from ar[ind] compare with its children & swap

while(2*i + 1 < N) { // Atleast 1 kid exists

int min_ind = 2*i + 1;

if((2*i + 2 < N) && (ar[2*i + 2] < ar[min_ind])) {

min_ind = 2*i + 2;

if(ar[i] > ar[min_ind]) {

swap ar[i] & ar[min_ind];

i = min_ind;

else { break; }

Priority Queue: Arranges data based on their priority values → Set/Min/Max

Functions:

1. add() : add element in Priority Queue : $\log(N)$: N = no of elements
2. peek() : return peek element : $O(1)$: peek: highest priority
3. remove() : removes peek element : $\log N$: N = no of elements
4. size() : returns size : $O(1)$

min PQ: Priority based on min values: {Min value at Top $\Rightarrow$ MinHeap}

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.add(19); ✓
```

```
pq.add(10); ✓
```

```
pq.add(15); ✓
```

```
pq.add(18); ✓
```

```
Print(pq.peek()); : 10
```

```
pq.remove(); : remove 10
```

```
Print(pq.peek()); : 15
```

```
Print(pq.peek()); : 15
```

pq:

19	10	15	18
----	---------------	----	----

Note: If want own ordering, override comparator

Added Code Link: Please refer it.

Problem 1 Connecting Ropes:

We are given an array that represents different size of ropes. In a single operation, you can connect two ropes.

Cost of connecting two ropes is sum of length of ropes you are connecting. Find the minimum cost of connecting all the ropes into 1.

$\{n, arr\} =$	Cost:	$\{n, arr\} =$	Cost:
$\{2, [5, 3, 2, 6]\} \rightarrow 6, 5$	11	$\{2, [2, 5, 3, 2, 6]\} \rightarrow 2, 2$	4
$\{2, [3, 2, 11]\} \rightarrow 3, 2$	5	$\{4, [5, 3, 6]\} \rightarrow 3, 4$	7
$\{2, [5, 11]\} \rightarrow 5, 11$	16	$\{7, [5, 6]\} \rightarrow 5, 6$	11
$\{2, [16]\} \rightarrow 2, 16$	18	$\{7, [11]\} \rightarrow 7, 11$	18
$\{18\} \rightarrow$	<u>Total Cost: 50</u>	$\{18\} \rightarrow$	<u>Total Cost: 40</u>

Idea: In each iteration, select 2 min ropes & cost it
Combine 2 ropes add in the rope list
Repeat till we have 1 rope.

Operations: getMin() deleteMin() add()

Data structures: MinHeap

0 1 2 3 4
 $arr = \{2, 5, 3, 2, 6\}$

Min-Heap	^{or} $getMin(): deleteMin()$	$getMin(): deleteMin()$	Insert
2 5 3 2 6	2	2	4
4 5 3 6	3	4	7
7 5 6	5	6	11
7 11	7	11	18
18	Stop & return Cost =		<u>Total Cost = 40</u>

int Min_Cost(int arr[]) {

PriorityQueue<Integer> pq = new PriorityQueue<>();

int N = arr.length;

for (int i = 0; i < N; i++) {

 pq.add(arr[i]);

}

int c = 0;

while (pq.size() > 1) {

 int a = pq.peek();

 pq.remove();

 int b = pq.peek();

 pq.remove();

 c += (a + b);

 pq.add(a + b);

}

return c;

}